

Implémenter un réseau pair à pair à l'aide des couplages

Simon Roux

TIPE - MPI

7 juin 2026

Introduction - Partage d'un fichier F dans un réseau

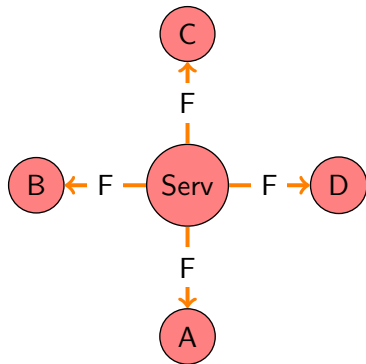


Figure – Partage d'un fichier par un serveur

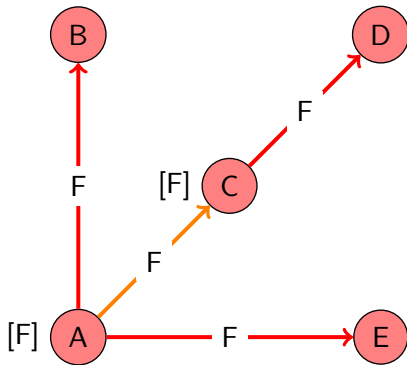
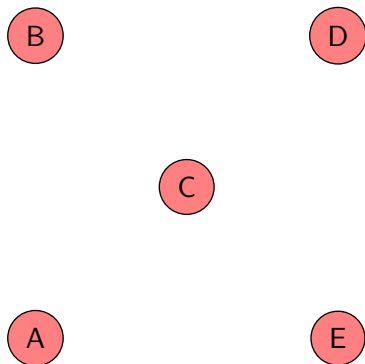


Figure – Partage d'un fichier dans un réseau P2P

Définition 1 - Réseau



Réseau : Ensemble d'utilisateurs (=noeuds)

Définition 2 - b-couplage

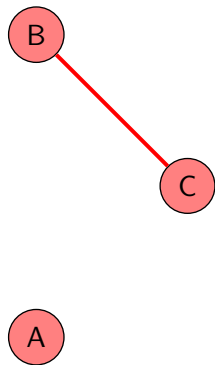


Figure – Cas $b = 1$

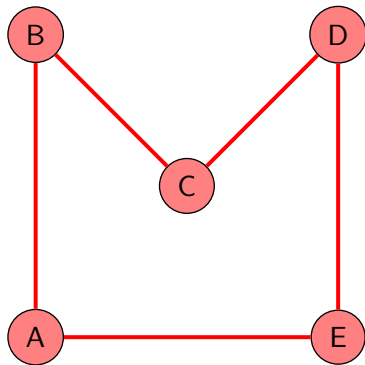


Figure – Cas $b = 2$

b -couplage : ensemble d'arêtes vérifiant $\forall n \in N, \#\{\{n, p\} | p \in N\} \leq b$.

Définition 3 - Listes de préférences

[C ; A ; E ; D] 

 [C ; E ; A ; B]

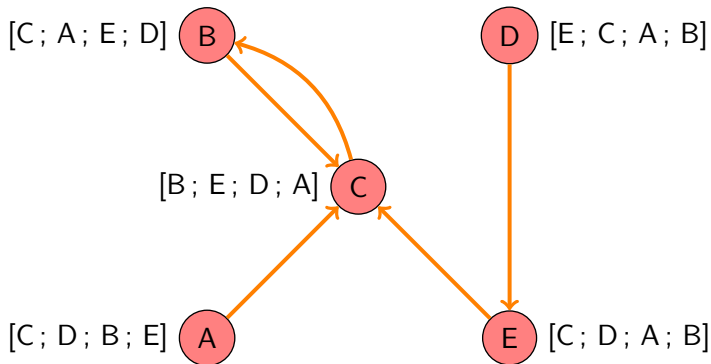
[B ; E ; D ; A] 

[C ; D ; B ; E] 

 [D ; C ; A ; B]

La **liste de préférences** d'un noeud ordonne les autres noeuds

Définition 4 - Graphe de préférences



Graphe de préférences : Sommets = utilisateurs, Arête de X à Y si Y est premier dans la liste de préférence de X

Graphe cyclique : s'il y a un k -cycle pour $k \geq 2$

Définition 5 - b-couplage stable

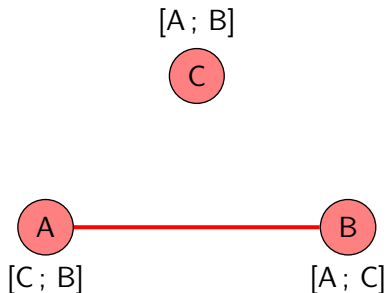


Figure – Couplage non stable

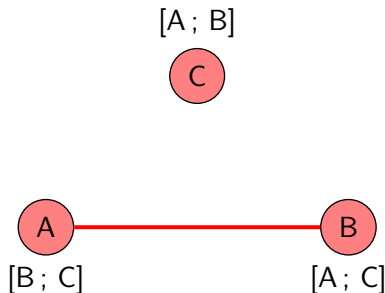


Figure – Couplage stable

$R_n(p)$: Rang du noeud p dans la liste de préférence de n .

b-couplage stable : Couplage C de cardinal maximal vérifiant

$\forall \{n, p\} \in C, \forall \{n', p'\} \in N^2, R_n(p) > R_n(p')$ ou $R_p(n) > R_p(n')$.

Pertinence les b-couplages stables

- Optimiser¹ les transferts de fichiers = déterminer un b-couplage stable
- Pourquoi les b-couplages stables ?
 - Donne le même rôle à tous les noeuds
 - Choix effectués localement par les noeuds

1. Au sens des listes de préférences

Proposition

Soit $b \in \mathbb{N}$ et R un réseau. Si le graphe de préférence de R est acyclique, alors R admet un b -couplage stable.

Dans un graphe de préférence acyclique :

- Implémentation pour un réseau statique
- Première adaptation de l'algorithme pour un réseau dynamique
- Deuxième adaptation moins naïve

Validation des résultats via temps d'activité des liaisons

Calcul de couplage stable statique²

Soit n, p deux utilisateurs

Définition

La paire $\{n, p\}$ est bloquante si n ou p n'est pas complètement couplé, ou si n et p préféreraient être couplés ensemble.

Algorithm 1 Calcul b-couplage stable statique

```
1: procedure STATIQUE( $r, b$ )           ▷ Un réseau non couplé  $r$ , et  $b \in \mathbb{N}$ 
2:   while Il existe une paire bloquante  $\{n, p\}$  do
3:     Si nécessaire, découpler le pire partenaire de  $n$  (resp.  $p$ )
4:     Coupler  $n$  et  $p$ 
5:   end while
6: end procedure
```

[1]

Couplage stable statique - Résultats

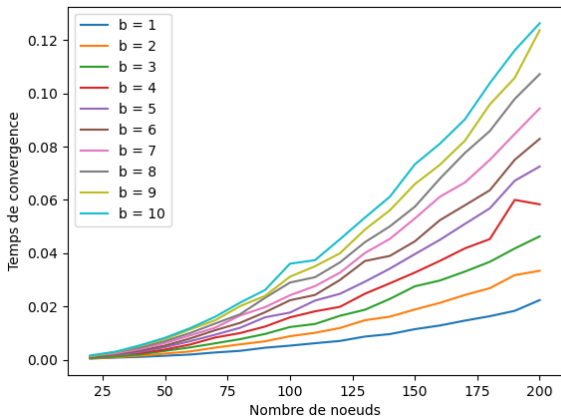


Figure – Temps de convergence de *STATIQUE*

Couplage stable statique - Résultats

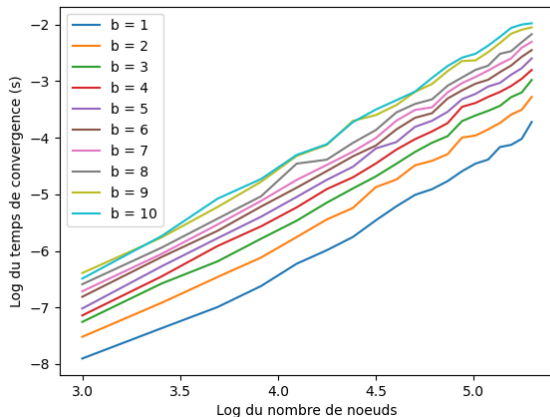


Figure – Logarithme du temps de convergence de *STATIQUE*

Complexité : $O(C_b n^a)$ où $a \simeq 2$

Couplage stable dynamique naïf

Description de l'algorithme :

- Construction d'un b-couplage stable
- A chaque modification, calcul d'un nouveau b-couplage stable via *STATIQUE*

Couplage stable dynamique naïf - Résultats

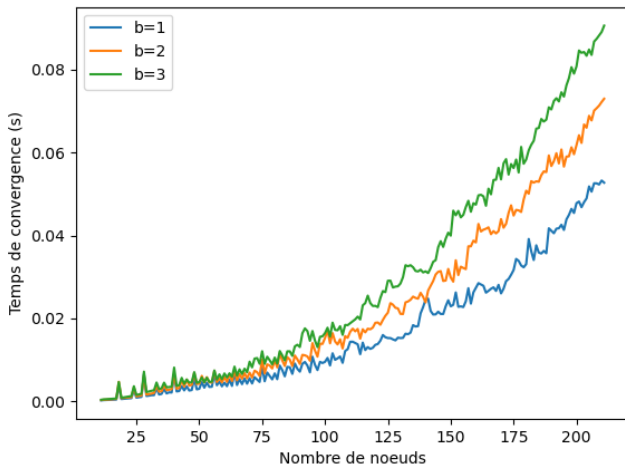


Figure – Ajouts successifs de 200 noeuds à partir de $|R| = 10$

Calcul de couplage stable dynamique par modification locale

Description de l'algorithme :

- Construction d'un b-couplage stable
- A chaque modification, changement local du couplage autour de la modification

Mesure de satisfaction³ pour un noeud i :

$$S_i = \frac{1}{b} \cdot (c_i + \frac{\sum_j R_j - C_j}{L_j}) \in [0, 1] \quad (1)$$

La satisfaction quantifie la stabilité du couplage.

3. Georgiadis & Papatriantafilou, 2012

Méthode de modification locale - Mesure de satisfaction

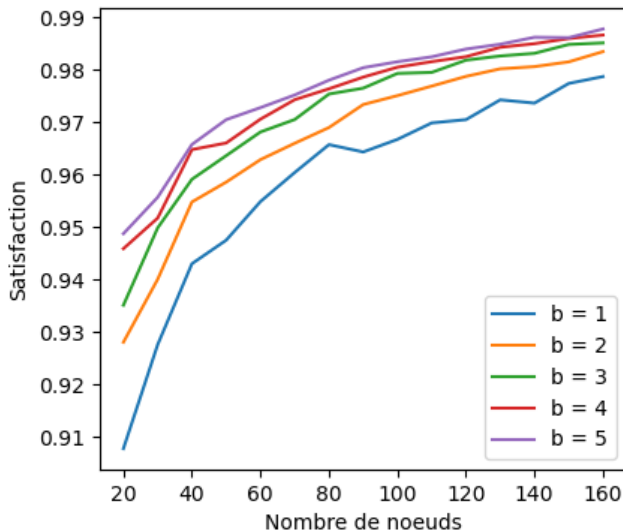


Figure – Satisfaction d'un couplage obtenu de *STATIQUE*

Méthode de modification locale - Résultats

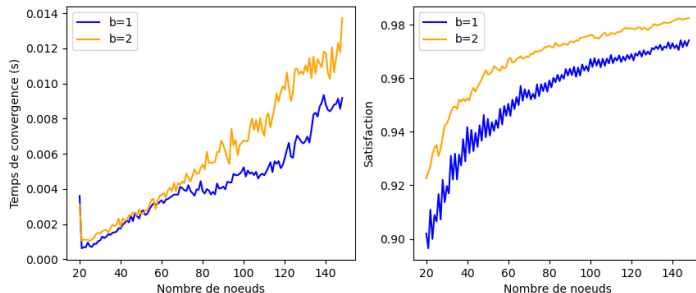


Figure – Ajouts successifs de 130 noeuds à partir de $|R| = 20$

Méthode de modification locale - Résultats

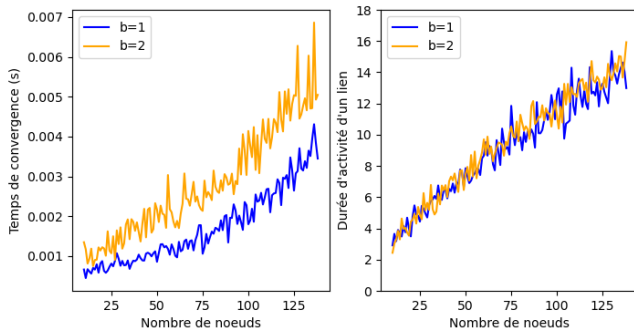


Figure – Ajouts et retraits successifs à $|R|$ fixé

- L'implémentation par couplages converge suffisamment rapidement
- La notion de satisfaction convient mieux que celle de couplage stable
- Perspectives :
 - Implémenter un algorithme réellement utilisé par les réseaux P2P → Comparaison
 - Démontrer un facteur d'approximation pour l'algorithme de modification locale

- [1] B. Cohen, *The BitTorrent Protocol Specification*, BitTorrent.org : BEP 3, 2008
- [2] D. Lebedev et al., *On Using Matching Theory to Understand P2P Network Design*, International Network Optimization Conference, 2007
- [3] F. Mathieu, *Self-stabilization in preference-based systems*, Discrete Applied Mathematics, 2007
- [4] G. Georgiadis, M. Papatriantafilou *Adaptive distributed b-matching in overlays with preferences*, International Symposium on Experimental Algorithms, 2012

$$S = \sum_i S_i = \sum_i \frac{1}{b} \cdot (c_i + \frac{\sum_j R_j - C_j}{L_i}) \in [0, 1] \quad (2)$$

c_i : Nombre de noeuds couplés à i

L_i : Longueur de la liste de préférence de i

R_j : Rang de j dans la liste de préférence de i

C_j : Rang de j parmi les noeuds couplés avec i

Annexe 2.1 - Condition d'acyclicité

Le graphe de préférence est désormais cyclique

Description de l'algorithme :

- On retient les paires bloquantes parcourues
- Si on détecte un cycle, suppression d'un arête du cycle
- Si un noeud n'a plus d'arêtes, il est supprimé

Annexe 2.2 - Condition d'acyclicité

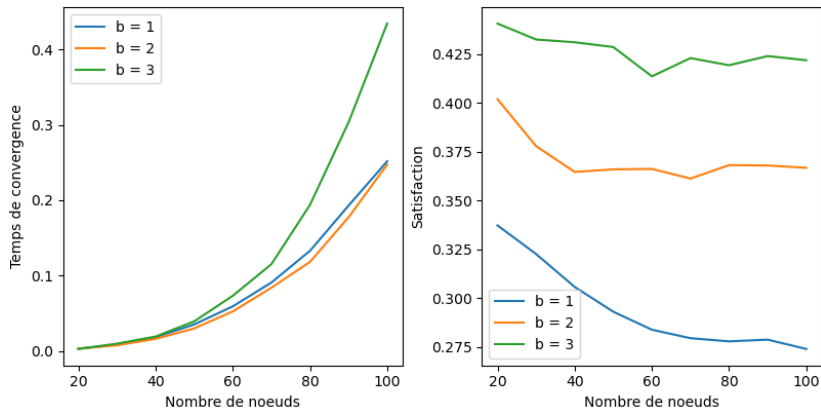


Figure – Temps de convergence (s) et satisfaction de l'adaptation aux graphes de préférences cycliques

Annexe 3.1 - Code principal

```
type trileen = Unmatched | Unloving | Loving
(*Différents états d'un noeud au sens d'une configuration*)

type node = {
  id: int;
  mutable ind_noeuds : int;
  (*indice du node dans le tableau reseau.noeuds *)
  mutable ind_unloving : int;
  (*indice du node dans reseau.unloving*)
  config : pfile;
  (* Tableau de noeuds rangé par marque décroissante des
  nodes avec lesquels le node peut échanger des données.
  C'est la liste d'adjacence de node dans le graphe
  d'acceptance. Les cases non utilisées sont à la fin du
  tableau et à None*)
```



```

couplage : pfile;
    (*Tableau de taille b ordonné par marque décroissante des
    nodes intervenant dans le couplage*)
mutable num_loving : int;
    (*Nombre (majoré par b et par couplage.len) de peers au
    sein de couplage qui sont loving*)
}

and pfile = {
    (*Tableau trié selon la marque décroissante. Les cases non
    utilisées sont à None*)
    mutable tab : champ option array;
    mutable len : int;
}

and champ = { (*Modélise une arête depuis un noeud*)
    n : node;
    e : trileen;
    (*indique si node est dans le couplage ou pas*)
    marque : int;}

```

```
type reseau = {  
    mutable noeuds : node option array;  
    (*tableau des nodes dans le réseau. Le node d'indice  
    node.id est à l'indice node.id*)  
    mutable len_noeuds : int;  
    (*Taille de noeuds*)  
    mutable len_unloving : int;  
    (*Taille de unloving. Fin de l'algo quand ça atteint 0*)  
    mutable unloving : node option array;  
    (*tableau des noeuds dont le couplage n'est pas composé  
    que de peers loving. La structure est la même que  
    le tableau noeuds*)  
}
```

```

let couplement node peer reseau marque =
(*On a trouvé une paire devant être couplée:
   On ajoute la paire au couplage et si la paire est
   loving on retire de chaque config le pair correspondant
   et on les enlève du unloving du réseau et on réduit le
   unloving du reseau si ils sont complètement couplés avec
   des loving*)
pfile_inserere node.couplage peer Unloving marque;
pfile_inserere peer.couplage node Unloving marque;
set_flag_in_pfile node.config peer Unloving None;
set_flag_in_pfile peer.config node Unloving None;
match peer.config.tab.(peer.config.len -1) with
| None -> (*Le peer choisi n'a pas de config\n"*)
    exit(1)
| Some c when c.n.id = node.id -> (*La paire est loving*)
    traitement_loving node peer reseau
| _ -> ()

```

```
let traitement_loving node peer reseau =  
  (*On effectue les démarches nécessaires pour coupler  
    définitivement node et peer*)  
  node.num_loving <- node.num_loving +1;  
  peer.num_loving <- peer.num_loving +1;  
  set_flag_in_pfile node.couplage peer Loving None;  
  set_flag_in_pfile peer.couplage node Loving None;  
  if node.num_loving = !b then (  
    (*Le node est complètement couplé qu'avec des paires  
      incassables: on le retire de unloving*)  
    nullify_unloving reseau node;  
    remove_from_couplages reseau node;)  
  if peer.num_loving = !b then (  
    nullify_unloving reseau peer;  
    remove_from_couplages reseau peer;)  
  rm_config node peer;  
  rm_config peer node
```

```

let protocol reseau =
  num.num <- 0;
  while reseau.len_unloving > 0 do
    initiative_best reseau;
  done

let connections_init p reseau =
  (*Génère une liste de préférence
  selon erdos-renyi*)
  let pf = pfile_vide () in
  for i = 0 to reseau.len_unloving -1 do
    match reseau.unloving.(i) with
    | None -> failwith "None au milieu du unloving"
    | Some node ->
      let m = rand_with_prob p in
      pfile_insere pf node Unmatched m;
  done;
  pf

```